

Clue Selection Under Noise

Codenames spymaster, baseline model

1 Introduction

A clue is a clue word together with a number k . Whenever we give one, our teammate ranks the unrevealed board words by how well each fits the clue word, and chooses words from the top, stopping when they reach k words or when they choose a word that is not ours, whichever comes first.

Suppose we and our teammate order words by cosine similarities from the same embedding space. Then choosing the optimal clue is just a search over the clue vocabulary for the word with the longest starting run of our team's words in the list sorted by decreasing similarity. In this case, our teammate chooses only the words that we intend, and never chooses a word that is not ours. The search is a matter of enumerating the vocabulary, and its solution is usually not unique.

However, in practice, our perceived similarities are related to but different from our teammate's, which changes the algorithm by which we find the optimal clue. The problem becomes probabilistic rather than deterministic, and we now have to account for the possibility of our teammate choosing a neutral, opponent, or assassin word. This paper describes one algorithm for finding an optimal clue under these conditions.

2 Setup

Let A be the set of our team's board words and B the set of all other board words. For any fixed clue word, let z_x be its similarity to board word x , standardized per clue word so that its similarities across the full board vocabulary have mean 0 and variance 1. Throughout this paper, i indexes A and w indexes B .

In order to construct an algorithm, we first define rewards — suppose selecting a word of A is worth 1, and selecting $w \in B$ costs $c_w > 0$. We want to choose the clue word and number k , with $1 \leq k \leq K$ for a fixed maximum K , that maximizes the expected reward.

To model deviations in the guesser's perceived similarities of the words from ours, we assume the guesser perceives board word x 's similarity with the clue word as $\hat{z}_x = z_x + \varepsilon_x$, where the ε_x are independent $\mathcal{N}(0, \sigma^2)$ for some free parameter σ . We also assume the guesser selects unrevealed words in decreasing order of $\hat{z}$, stopping after k selections or upon selecting a word of B , whichever comes first.

Let

$$T = \max_{w \in B} \hat{z}_w, \quad W = \arg \max_{w \in B} \hat{z}_w, \quad N = \#\{i \in A : \hat{z}_i > T\}.$$

In words, W is the non-team word with the highest similarity from the guesser, T is

the guesser's similarity of that word, and N is the number of team words that the guesser ranks higher than W .

We know the guesser reveals $\min(k, N)$ words of A , and selects a word of B (that word being W) if and only if $N < k$. Note that no word of B other than W can be selected. Thus, our goal is to maximize $G(k) - P(k)$, where

$$G(k) = \mathbb{E}[\min(k, N)], \quad P(k) = \mathbb{E}[c_W \mathbf{1}\{N < k\}].$$

3 Derivation

Neither G nor P has a closed form. Both are expectations over all $|A| + |B|$ noise variables at once, and the maximum of non-identical normals has no elementary distribution, so the aim is not to solve them exactly but to reduce them far enough to evaluate cheaply.

Our goal is to reduce both expressions to elementary operations and functions of z_x , the standard normal PDF and CDF, and single integrals.

We start with $G(k)$. Since $\min(k, N) = \sum_{j=1}^k \mathbf{1}\{N \geq j\}$, taking expectations and conditioning on T gives

$$G(k) = \sum_{j=1}^k \int \Pr(N \geq j \mid T = t) dF_T(t).$$

To evaluate this, we need the distribution of T and the distribution of N given $T = t$.

For the first, let φ and Φ be the PDF and CDF of a standard normal, so that the PDF and CDF of $\hat{z}_w \sim \mathcal{N}(z_w, \sigma^2)$ are

$$f_{\hat{z}_w}(t) = \sigma^{-1} \varphi\left(\frac{t - z_w}{\sigma}\right), \quad F_{\hat{z}_w}(t) = \Phi\left(\frac{t - z_w}{\sigma}\right).$$

Since T is a maximum of independent variables,

$$F_T(t) = \prod_{w \in B} F_{\hat{z}_w}(t) = \prod_{w \in B} \Phi\left(\frac{t - z_w}{\sigma}\right).$$

For the second, note that T is a function of $(\varepsilon_w)_{w \in B}$ only, which is independent of $(\varepsilon_i)_{i \in A}$. Thus conditioning on $T = t$ does not change the distribution of the $\hat{z}_i$, and each $i \in A$ independently satisfies $\hat{z}_i > t$ with probability

$$p_i(t) = \Phi\left(\frac{z_i - t}{\sigma}\right).$$

Thus, given $T = t$, N is a sum of $|A|$ independent Bernoulli variables, the i -th with success probability $p_i(t)$.

Now we move on to $P(k)$. Note that if we fix $T = t$, then N is solely a function of $(\varepsilon_i)_{i \in A}$, whereas W is solely a function of $(\varepsilon_w)_{w \in B}$. Thus, c_W and $\mathbf{1}\{N < k\}$ are conditionally

independent given T , so

$$P(k) = \int \Pr(N < k \mid T = t) \bar{c}(t) dF_T(t), \quad \bar{c}(t) = \mathbb{E}[c_W \mid T = t].$$

We have already derived the distribution of T and the first factor, so we just need $\bar{c}(t)$, which requires $\Pr(W = w \mid T = t)$.

Writing g_w for the joint density of $W = w$ and T at t ,

$$\Pr(W = w \mid T = t) = \frac{g_w(t)}{f_T(t)}.$$

$W = w$ and $T = t$ means $\hat{z}_w = t$ and $\hat{z}_v < t$ for every other $v \in B$. The $\hat{z}$ are independent, so

$$g_w(t) = f_{\hat{z}_w}(t) \prod_{v \neq w} F_{\hat{z}_v}(t).$$

The events $\{W = w\}$, $w \in B$, are disjoint and exhaustive, so

$$f_T(t) = \sum_{w \in B} g_w(t).$$

Since $F_T(t) = \prod_{v \in B} F_{\hat{z}_v}(t)$, the product over $v \neq w$ is $F_T(t)/F_{\hat{z}_w}(t)$. Set

$$h_w(t) = \frac{f_{\hat{z}_w}(t)}{F_{\hat{z}_w}(t)} = \frac{\sigma^{-1} \varphi((t - z_w)/\sigma)}{\Phi((t - z_w)/\sigma)},$$

so that $g_w(t) = F_T(t) h_w(t)$ and $f_T(t) = F_T(t) \sum_v h_v(t)$. The $F_T(t)$ cancels in the ratio:

$$\Pr(W = w \mid T = t) = \frac{h_w(t)}{\sum_v h_v(t)}, \quad \bar{c}(t) = \frac{\sum_w c_w h_w(t)}{\sum_w h_w(t)}.$$

4 Numerical evaluation

Both G and P have the form $\int \psi(t) dF_T(t)$ for a bounded ψ :

$$\psi_G(t) = \sum_{j=1}^k \Pr(N \geq j \mid T = t), \quad \psi_P(t) = \Pr(N < k \mid T = t) \bar{c}(t).$$

Both are evaluated by numerical integration. Partition an interval $[t_0, t_M]$ into cells $[t_m, t_{m+1}]$, take ψ at the midpoint $\bar{t}_m = \frac{1}{2}(t_m + t_{m+1})$ of each, and weight it by the probability that T lands in that cell:

$$u_m = F_T(t_{m+1}) - F_T(t_m), \quad \int \psi(t) dF_T(t) \approx \sum_m \psi(\bar{t}_m) u_m.$$

Because the range of T is $\mathbb{R}$, we have to limit our integral approximation to bounds t_0 and t_M . Take t_0 below every z_w and t_M above every z_w , each by several σ , so that $F_T(t_0)$ and $1 - F_T(t_M)$ are negligible.

Computing $\tilde{c}(t)$ takes $O(|B|)$.

The probabilities $\Pr(N \geq j \mid T = t)$ follow from the standard recursion for a sum of independent Bernoulli variables. Letting $q_r^{(i)}$ be $\Pr(N = r)$ after the first i words of A ,

$$q_0^{(0)} = 1, \quad q_r^{(i)} = q_r^{(i-1)}(1 - p_i(t)) + q_{r-1}^{(i-1)} p_i(t),$$

and $\Pr(N \geq j \mid T = t) = 1 - \sum_{r < j} q_r$. Counts above K are never used, so this takes $O(|A|K)$.

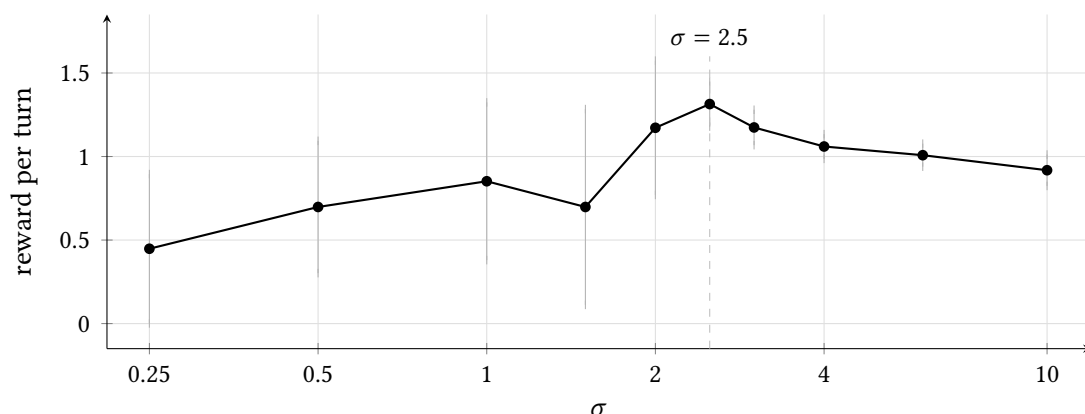
The total is $O(M(|A|K + |B|))$ per clue, with M the number of cells.

5 Results

The model above has one free parameter. To choose it, σ was swept over three orders of magnitude and each value was made to play the same 100 positions, drawn from held-out board words and spanning openings through endgames. Each turn was scored as the game scores it: the guesser's ranking is walked, own words are counted until the first word of B , and the reward is +1 per own word with -0.2 , -1 and -10 for a neutral, opponent and assassin word respectively. The guesser was Claude Sonnet, prompted to rank the unrevealed board words by how related each is to the clue word.

σ	announced	revealed	gap	reward	s.e.	asn	opp	neu	clean
0.25	3.96	1.55	-2.41	0.448	0.228	5	53	36	6
0.5	3.91	1.60	-2.31	0.698	0.202	3	53	36	8
1.0	3.45	1.74	-1.71	0.852	0.241	4	43	29	24
1.5	2.79	1.72	-1.07	0.698	0.299	7	28	21	44
2.0	2.33	1.69	-0.64	1.172	0.205	3	19	14	64
2.5	1.72	1.41	-0.31	1.314	0.069	0	7	13	80
3.0	1.40	1.22	-0.18	1.174	0.054	0	3	8	89
4.0	1.21	1.10	-0.11	1.060	0.038	0	3	5	92
6.0	1.16	1.05	-0.11	1.008	0.035	0	3	6	91
10.0	1.13	0.99	-0.14	0.918	0.048	0	6	6	88

Per turn, over 100 positions. *announced* is k ; *revealed* is the number of words of A the guesser actually took; the last four columns count how each turn ended.



Realized reward per turn. Bars are 95% confidence intervals; the wide bars at small σ are assassin turns, each costing -10 .

This sweep selects $\sigma = 2.5$, and on its own terms that is the right answer: reward per turn peaks there, and the interval is narrow because turns at that setting almost never end on the assassin.

6 The proxy disagrees with the game

Reward per turn is a proxy. The quantity we actually care about is whether the model wins, so each setting was then made to play full two-team games against a fixed opponent — the centroid baseline — over 100 games per setting, each board played twice with the sides swapped so that the first-move advantage cancels. The guesser was Claude Sonnet on both sides.

σ	win %	announced k	own words/turn	reward/turn
1.0	69	2.67	1.48	0.85
1.25	70	2.27	1.43	1.01
1.5	70	1.93	1.33	1.01
2.0	57	1.42	1.17	1.05
2.5	51	1.16	1.07	0.98

100 games per setting against the centroid baseline.

The ordering is nearly reversed. The setting the proxy chose, $\sigma = 2.5$, wins 51% of its games — indistinguishable from the baseline it is supposed to beat — while σ between 1.0 and 1.5 wins about 70%.

The reason is visible in the last two columns, and it is not that the proxy was measured badly. Reward per turn is genuinely flat: every setting from 1.25 upward sits between 0.98 and 1.05, because the additional words a bold setting wins are very nearly cancelled by the opponent words and assassins it also concedes. A metric that does not

vary cannot select anything, and the peak at 2.5 in Section 5 is within the noise of that flatness.

What does vary is words per turn, which falls monotonically from 1.48 to 1.07 as σ grows. Codenames is a race: both teams are emptying their own half of the board, and a team taking 1.48 words per turn finishes in roughly two-thirds the turns of one taking 1.07. Per-turn reward prices a turn in isolation and is blind to how many turns the game will last, so it cannot see the cost of playing slowly. Only $\sigma = 1.0$ is punished on both measures, where a 14% assassin rate at -10 drags reward per turn down to 0.85 without buying a better win rate than 1.25 or 1.5.

We therefore take $\sigma = 1.5$: the top of the win-rate plateau, and the cautious end of it, since 1.0 and 1.25 win no more often while ending turns on the assassin more than twice as frequently.

The general lesson is worth stating separately from this model. A per-turn proxy was used because it is cheap — one ranking per position rather than a full game — and it selected a parameter that plays the eventual objective close to chance. The proxy was not wrong about what it measured; it was measuring a quantity the game does not maximize.